VR-Sets-ZFA

Operational Reference Semantics for Non-Well-Founded Sets

Vitaly Reznik

Version 1.0.1 — 31 May 2026

Abstract

VR-Sets-ZFA is a foundational extension of VR-Sets providing operational reference semantics for non-well-founded sets in Lean 4. The construction takes the coinductive parallel of mathlib's inductive PSet via PFunctor.M, defines an extensional cobisimulation quotient OSetZFA, proves Aczel's Anti-Foundation Axiom (AFA) as a theorem from the final coalgebra property, and establishes a faithful embedding OSet → OSetZFA of well-founded sets into the ZFA universe.

The work formally resolves Conjecture IV.2 from VR-Sets: a type satisfying AFA can be constructed in VR's operational universe. The Quine atom and an omega chain are machine-verified non-vacuity witnesses; the bisimulation collapse identifying the two-cycle graph with the self-loop graph is proved as theorem cycleDecoration_eq_quineAtom.

Sixth work in the VR Cycle. Companion to the Lean 4 formalisation: DOI 10.5281/zenodo.20368268.

* * *

Part I — Position

I.1 Place in the VR Cycle

VR-Sets-ZFA is the sixth work in the VR Cycle, following VR (A Formal System), VR-Numbers, VR-Sets, VR-Forms, and VR-Audit. The first four works are foundational; the fifth is applied. VR-Sets-ZFA is foundational: it extends the operational universe of VR-Sets to admit non-well-founded sets through a new coinductive type, while preserving compatibility with the existing well-founded construction via faithful embedding.

The work is not an audit. It is not the application of existing apparatus to a classical theorem (as VR-Audit-1 was for Hahn–Banach). It constructs new foundational types and proves theorems about them. The wrapping principle of VR-Audit does not apply: there is no mathlib infrastructure for non-well-founded sets to wrap. The work builds the infrastructure itself, using mathlib's PFunctor.M as a substrate.

I.2 Formal answer to Conjecture IV.2

VR-Sets posed (in Conjectures.lean) the question of whether a type satisfying Aczel's Anti-Foundation Axiom can be constructed in VR's operational universe. The conjecture was left open because the Lean implementation of VR-Sets chose inductive PSet as the foundation: PSet.mem_irrefl and PSet.mem_wf follow from inductive structure, making ZFA-style non-well-foundedness impossible in OSet. The boundary witnesses AFA_Refuted and quineAtom_impossible are explicit in Modes.lean.

VR-Sets-ZFA answers Conjecture IV.2 constructively. The type OSetZFA = Quotient CoPSet.cobisim, where CoPSet is the coinductive parallel of PSet via PFunctor.M, satisfies AFA. The proof is direct: AFA_in_OSetZFA is established by exhibiting the unique decoration of each graph via M.corec, with uniqueness shown by extensional cobisimulation. The Quine atom is constructed concretely as the decoration of the single-vertex self-loop graph.

I.3 What this work is not

VR-Sets-ZFA is not a replacement for VR-Sets. The existing well-founded OSet construction is unchanged. OSet remains the foundation for VR-Numbers and other VR Cycle works. VR-Sets-ZFA sits beside OSet as an extension, with OSet embedding faithfully into OSetZFA via embedOSet.

VR-Sets-ZFA is not a contribution to mathlib's ZFA infrastructure. Mathlib has no ZFA; this work is implemented in the VR Cycle namespace and does not propose upstream changes. External users obtain VR-Sets-ZFA by depending on the VR Cycle, not by importing from mathlib.

VR-Sets-ZFA is not a comprehensive treatment of non-well-founded set theory. It establishes AFA and demonstrates non-vacuity, but does not formalise process algebra, modal logic, coalgebraic semantics, or other AFA applications. These remain as candidate future work.

VR-Sets-ZFA addresses Aczel's AFA only. Alternative anti-foundation axioms (Scott's SAFA, Boffa's BAFA, Finsler's FAFA) are not considered.

* * *

Part II — Architecture

II.1 Two parallel constructions

VR-Sets has the construction chain:

```
PSet := inductive mk (α : Type u) (A : α → PSet)

PSet.Equiv x y : mutual simulation of membership (well-founded recursion)

ZFSet := Quotient PSet.setoid

OSet := ZFSet
```

VR-Sets-ZFA has the parallel construction:

```
CoPSetFunctor : PFunctor := ⟨Type u, id⟩

CoPSet := PFunctor.M CoPSetFunctor

CoPSet.Equiv x y : mutual cosimulation of membership (coinductive)

OSetZFA := Quotient CoPSet.cobisim_setoid
```

The structural parallelism is exact: inductive PSet corresponds to coinductive CoPSet via final coalgebra duality. PSet uses well-founded recursion for equality; CoPSet uses extensional bisimulation. Both pass through a quotient by their respective equivalence to obtain a Lean type with proper extensional equality.

II.2 The polynomial functor

The polynomial functor underlying CoPSet is the same shape as the constructor for PSet. In PFunctor notation:

```
CoPSetFunctor.A = Type u    -- shape type (indices of children)

CoPSetFunctor.B = id        -- children indexed by their shape
```

A CoPSet element x decomposes as ⟨α, A⟩ where α : Type u is the shape and A : α → CoPSet is the children family. This decomposition is exposed through CoPSet.dest : CoPSet → Σ α : Type u, α → CoPSet, with projections CoPSet.shape and CoPSet.children.

The universe arithmetic: CoPSetFunctor : PFunctor.{u+1, u}, and PFunctor.M CoPSetFunctor : Type (u+1). This matches PSet.{u} : Type (u+1) exactly, making CoPSet a drop-in coinductive substitute at the same universe level.

II.3 PFunctor.M as the construction substrate

Mathlib's PFunctor.M (Avigad–Carneiro–Hudon 2019) provides the greatest fixpoint of a polynomial functor as a structure of convergent approximation sequences. The key operations available without further work are:

```
M.mk      : F (M F) → M F           (constructor)

M.dest    : M F → F (M F)           (destructor)

M.corec   : (X → F X) → X → M F     (corecursor)

M.bisim   : strong bisimulation principle

M.corec_unique : universal property of M.corec
```

M.corec and M.corec_unique together provide what is called the universal property of the final coalgebra: every coalgebra (X, X → F X) admits a unique map into M F respecting the coalgebra structure. This is precisely the content of Aczel's AFA when F is the set-shape functor. The work of

VR-Sets-ZFA is to transit this property through the extensional quotient and verify that it survives there.

II.4 Strong versus extensional bisimulation

M.bisim is a strong bisimulation principle: two M-elements are equal if related by a relation R such that, for each related pair (x, y), the shapes coincide as types and children are R-related index-by-index. This requires shape equality at the type level, not merely cardinality equivalence.

Set-theoretic equality is extensional: two sets are equal if they have the same members, regardless of how the members are indexed. A set {∅} can be presented as the M-element of shape Bool with both indices mapping to ∅, or as the M-element of shape Unit with the unique index mapping to ∅. These are strongly bisimilar in the M-element sense (different shapes), so M.bisim does not collapse them. They are extensionally bisimilar — both represent the singleton {∅} as a set.

VR-Sets-ZFA's extensional bisimulation relation, CoPSet.Equiv, captures this:

```
def CoPSet.isBisim (R : CoPSet → CoPSet → Prop) : Prop :=
  ∀ x y, R x y →
    (∀ i : x.shape, ∃ j : y.shape, R (x.children i) (y.children j)) ∧
    (∀ j : y.shape, ∃ i : x.shape, R (x.children i) (y.children j))


def CoPSet.Equiv (x y : CoPSet) : Prop :=
  ∃ R : CoPSet → CoPSet → Prop, CoPSet.isBisim R ∧ R x y
```

The forward and backward clauses match children by existence, not by index identity. Different-shape representations of the same set are CoPSet.Equiv-related but not M.bisim-related. The quotient by CoPSet.Equiv is therefore non-trivial and produces a strictly different type from CoPSet itself.

II.5 The quotient OSetZFA

CoPSet.Equiv is proved reflexive, symmetric, and transitive — the standard setoid laws — via existential witnesses (identity, transpose, composition). The transitivity witness is the relational composition R_trans a c := ∃ b, R_1 a b ∧ R_2 b c, with isBisim closure checked by chaining forward/backward simulation steps.

The quotient is then standard:

```
instance CoPSet.instSetoid : Setoid CoPSet := ⟨Equiv, refl, symm, trans⟩

def OSetZFA := Quotient CoPSet.instSetoid
```

Equality in OSetZFA, written ≡_ZFA, is identical to cobisimulation by construction. The classical theorem of ZFA — that two sets are equal iff their picturing graphs are bisimilar — is definitional in OSetZFA, not derived.

* * *

Part III — AFA as Theorem

III.1 Statement

Aczel's Anti-Foundation Axiom, in operational form on OSetZFA:

```
theorem AFA_in_OSetZFA :
  ∀ (V : Type u) (E : V → V → Prop),
    ∃! f : V → OSetZFA, isDecoration V E f
```

where isDecoration is the extensional decoration predicate:

```
def isDecoration (V : Type u) (E : V → V → Prop) (f : V → OSetZFA) :
Prop :=
  ∀ v : V, ∀ x : OSetZFA, x ∈ f v ↔ ∃ w : V, E v w ∧ x = f w
```

The decoration condition says: f v has, as its members, precisely the f-images of v's E-neighbours. This is the standard set-theoretic content of AFA in the language of graph decoration.

III.2 Existence via M.corec

The decoration is built directly from the graph (V, E) as a coalgebra:

```
def graphCoalg (V : Type u) (E : V → V → Prop) (v : V) :=
  ⟨{w // E v w}, Subtype.val⟩ : PFunctor.Obj CoPSetFunctor V


noncomputable def graphCoPSet (V : Type u) (E : V → V → Prop) : V → CoPSet
:=
  PFunctor.M.corec (graphCoalg V E)


noncomputable def graphDecoration (V : Type u) (E : V → V → Prop) : V →
OSetZFA :=
  fun v => OSetZFA.mk (graphCoPSet V E v)
```

The shape at vertex v is the subtype {w // E v w} of E-neighbours; children are indexed by these neighbours and unfold corecursively. The destructor computation:

```
graphCoPSet_dest : (graphCoPSet V E v).dest =
  ⟨{w // E v w}, fun i => graphCoPSet V E i.val⟩
```

This is proved by rfl. PFunctor.M.dest_corec is definitional in mathlib's implementation, so the unfolding requires no explicit lemma manipulation.

The existence half of AFA — graphDecoration_isDecoration — follows by unfolding the decoration condition through graphCoPSet_dest and CoPSet.mem_mk.

III.3 Uniqueness via cobisimulation

M.corec_unique establishes uniqueness up to strict M-equality on the M-type level. This does not lift directly to OSetZFA, because two functions f, g : V → OSetZFA satisfying isDecoration may have CoPSet representatives that are extensionally bisimilar but not strongly bisimilar. The uniqueness in OSetZFA must be established at the extensional level.

The proof exhibits an explicit cobisimulation. Given f satisfying isDecoration, classical representatives fRep v : CoPSet are chosen (Classical.choose on OSetZFA.mk_surjective). The relation

```
R c d := ∃ v : V, CoPSet.Equiv c (fRep v) ∧ CoPSet.Equiv d (graphCoPSet V
E v)
```

is shown to be a bisimulation by chaining: a child of c corresponds via CoPSet.isBisim_Equiv to a child of fRep v, which by isDecoration corresponds to fRep w for some neighbour w, which by classical representative correspondence corresponds to a child of graphCoPSet V E v indexed by w (using graphCoPSet_dest). The backward direction is symmetric.

Then CoPSet.bisim_imp_Equiv yields CoPSet.Equiv (fRep v) (graphCoPSet V E v) for every v, whence OSetZFA.sound gives f v = graphDecoration V E v. The final theorem packages existence and uniqueness:

```
theorem AFA_in_OSetZFA (V : Type u) (E : V → V → Prop) :
    ∃! f : V → OSetZFA, isDecoration V E f :=
  ⟨graphDecoration V E,
   graphDecoration_isDecoration V E,
   fun f hf => graphDecoration_unique V E f hf⟩
```

III.4 AFA as theorem, not axiom

AFA is established as a theorem of VR-Sets-ZFA, not postulated as an axiom. The proof axioms remain within mathlib's standard ceiling: [propext, Classical.choice, Quot.sound]. No additional axioms are introduced.

This parallels Theorem III.9 (Choice) in VR-Sets, which proves the axiom of choice as a theorem in the operational universe using Classical.epsilon. In both cases, the principle in question becomes a theorem of the framework rather than a postulate alongside the framework. The architectural cost is borne by the underlying type construction (CoPSet via PFunctor.M, OSet via PSet); the theorem itself is short once the substrate is correct.

The methodological gain: VR-Sets-ZFA carries no axiom that classical ZFA + AFA would require. Foundational extension is achieved without weakening the axiom profile relative to mathlib.

* * *

Part IV — Embedding

IV.1 OSet inside OSetZFA

Well-founded sets embed into the ZFA universe via structural recursion on PSet:

```
def embedPSet : PSet → CoPSet
  | PSet.mk α A => CoPSet.mk α (fun i => embedPSet (A i))
```

This is an inductive definition (PSet is inductive; A i is a strict subterm of PSet.mk α A) producing a coinductive value. Lean accepts the recursion through structural decreasing on PSet.

The map descends to OSet through the quotient via Quotient.lift, requiring embedPSet to respect PSet.Equiv:

```
noncomputable def embedOSet : OSet → OSetZFA :=
  Quotient.lift (fun p => OSetZFA.mk (embedPSet p))
    (fun _ _ h => OSetZFA.sound (embedPSet_congr h))
```

The congruence lemma embedPSet_congr : PSet.Equiv x y → CoPSet.Equiv (embedPSet x) (embedPSet y) closes the lift.

IV.2 Faithfulness

Three theorems establish faithfulness of embedOSet:

```
theorem embedPSet_faithful :
  CoPSet.Equiv (embedPSet x) (embedPSet y) → PSet.Equiv x y


theorem embedOSet_injective : Function.Injective embedOSet


theorem embedOSet_mem (x a : OSet) :
```

```
embedOSet x ∈ embedOSet a ↔ x ∈ a
```

Together they say: distinct well-founded sets map to distinct ZFA sets; membership is preserved across the embedding. The well-founded subuniverse of OSetZFA — the image of embedOSet — is isomorphic to OSet as a set theory.

IV.3 Asymmetric proof techniques

The forward direction of congruence (PSet.Equiv → CoPSet.Equiv) and the backward direction of faithfulness (CoPSet.Equiv → PSet.Equiv) require different proof techniques. The asymmetry is fundamental to the inductive/coinductive divide.

Forward (embedPSet_congr) is proved by pure bisimulation, with no induction on PSet. The relation

```
R c d := ∃ x y : PSet, PSet.Equiv x y ∧ c = embedPSet x ∧ d = embedPSet y
```

is shown to be a CoPSet bisimulation in one structural decomposition step. CoPSet.bisim_imp_Equiv closes the goal coinductively. The coinductive framework absorbs the entire proof obligation.

Backward (embedPSet_faithful) requires structural induction on PSet. Given CoPSet.Equiv (embedPSet (PSet.mk α A)) (embedPSet (PSet.mk β B)), the children correspondence is extracted via CoPSet.isBisim_Equiv, and the induction hypothesis is applied at each child A i to obtain PSet.Equiv (A i) (B j) for some j. PSet's well-foundedness — its inductive structure — is the proof resource that makes this descent terminate.

The schematic observation: embedding a well-founded structure into a coinductive universe is bisimulation-free; extracting well-founded structure from coinductive equivalence requires well-foundedness as proof resource. The asymmetry tracks the direction of the inductive/coinductive shift.

IV.4 ZFC ⊊ ZFA strict inclusion

The embedding is faithful (injective, membership-preserving) but not surjective. The Quine atom is an explicit witness to non-surjectivity:

```
theorem quineAtom_not_in_range_embedOSet :
  quineAtom ∉ Set.range embedOSet
```

Proof: if embedOSet x = quineAtom, then x ∈ x (via embedOSet_mem and quineAtom_self_mem), contradicting ZFSet.mem_wf.

The map ZFC 🔜 ZFA is therefore a strict embedding, not an isomorphism. Non-well-founded sets are genuinely new objects in OSetZFA, not reformulations of well-founded sets.

* * *

Part V — Demonstrations

V.1 The Quine atom

The Quine atom is the canonical self-membered set, constructed as the decoration of the single-vertex self-loop graph:

```
noncomputable def quineAtom : OSetZFA :=
  graphDecoration (fun (_ _ : Unit) => True) ()


theorem quineAtom_mem_iff (z : OSetZFA) :
  z ∈ quineAtom ↔ z = quineAtom


theorem quineAtom_self_mem : quineAtom ∈ quineAtom :=
  (quineAtom_mem_iff quineAtom).mpr rfl
```

The membership characterisation reduces to: the only member of quineAtom is quineAtom itself. Self-membership is immediate. The canonical equation q = {q} is also expressible directly:

```
theorem quineAtom_eq_singleton_self :
  quineAtom = OSetZFA.singleton quineAtom
```

This is proved via OSetZFA.ext from quineAtom_mem_iff and mem_singleton.

V.2 Bisimulation collapse

Different-looking accessible pointed graphs can produce the same OSetZFA element. The two-vertex cycle graph (Bool, ≠) and the single-vertex self-loop graph (Unit, True) are both decorated to the Quine atom:

```
noncomputable def cycleDecoration : Bool → OSetZFA :=
  graphDecoration (fun x y : Bool => x ≠ y)


theorem cycleDecoration_eq_quineAtom (b : Bool) :
  cycleDecoration b = quineAtom
```

The proof uses CoPSet.bisim_imp_Equiv with the relation

```
R c d := (c = graphCoPSet (≠) false ∨ c = graphCoPSet (≠) true)
       ∧ d = graphCoPSet (fun _ _ : Unit => True) ()
```

Both nodes of the two-cycle and the single self-loop node have one child (themselves or each other, respectively), and after cobisimulation these are identified. The graph that the apparent two-vertex cycle describes is, extensionally, the same set as the self-loop.

This is a theorem of the construction, not a limitation. Extensional bisimulation correctly identifies these structures. Strong bisimulation (M.bisim) would distinguish them, since the shape types Bool and Unit are not equal. The choice of extensional quotient over strong bisimulation is precisely what makes OSetZFA a set theory rather than a tree theory.

V.3 Two modes of non-well-foundedness

Non-well-foundedness in OSetZFA admits two distinct modes. The Quine atom exemplifies self-membership:

```
q ∈ q ∈ q ∈ ⋯  (infinite chain by repetition)
```

The omega chain exemplifies infinite descent without self-membership:

```
noncomputable def omegaChain : ℕ → OSetZFA :=
  graphDecoration (fun n m : ℕ => m = n + 1)


theorem omegaChain_descent (n : ℕ) :
  omegaChain (n + 1) ∈ omegaChain n
```

Here the chain omegaChain 0 ∋ omegaChain 1 ∋ omegaChain 2 ∋ ⋯ has no self-membered element; each step descends to a distinct set. Both phenomena exist in OSetZFA, neither exists in OSet.

V.4 Non-well-foundedness of OSetZFA membership

Membership on OSetZFA is not well-founded:

```
theorem OSetZFA_mem_not_wf :
  ¬ WellFounded (· ∈ · : OSetZFA → OSetZFA → Prop)
```

Proof: if · ∈ · were well-founded, the Quine atom would be accessible, and acc_irrefl would contradict quineAtom_self_mem. This contrasts directly with OSet, where IsWellFounded (· ∈ ·) is a theorem inherited from mathlib's ZFSet via PSet's inductive structure.

The two membership relations therefore have categorically different proof-theoretic behaviour. Induction on membership is available in OSet; coinduction is available in OSetZFA. The embedding embedOSet does not transport well-foundedness, since OSetZFA contains witnesses (the Quine atom) that violate it.

* * *

Part VI — Methodological Observations

Observation 1 (M-type as ZFA substrate). Mathlib's PFunctor.M, intended as the general coinductive type former for polynomial functors, is directly usable as a set-shape coinductive type without modification. The polynomial functor ⟨Type u, id⟩ — shape is the index type, children are indexed by the shape — captures exactly the constructor of PSet. The construction of CoPSet is one line: def CoPSet := PFunctor.M ⟨Type u, id⟩. The universal property M.corec_unique is consumed as the AFA uniqueness theorem after the extensional quotient.

Observation 2 (strong vs extensional bisimulation). PFunctor.M.bisim is a strong bisimulation principle: it collapses bisimilar M-elements when they have identical shape types and index-by-index R-related children. Set-theoretic equality is extensional: shape types may differ as long as the multisets of children match. These are distinct relations on CoPSet, and the difference is non-trivial: two CoPSet elements representing the singleton {∅} with different shape types are extensionally equal but not strongly bisimilar. ZFA construction via PFunctor.M therefore requires an extensional quotient, not just the M-type's built-in bisim collapse.

Observation 3 (AFA as theorem via final coalgebra). On the extensional quotient OSetZFA, AFA emerges as a theorem from M.corec (existence) and a bisimulation argument lifting M.corec_unique to OSetZFA (uniqueness). The real work is in the construction of CoPSet and the cobisimulation quotient; the AFA proof itself is approximately 90 lines. This parallels Theorem III.9 Choice in VR-Sets becoming short through Classical.epsilon — work shifts from final theorem to underlying machinery.

Observation 4 (cobisimulation as definitional equality). Classical ZFA theory derives the result that two sets are equal iff their picturing graphs are bisimilar. In OSetZFA this is definitional: a ≡_ZFA b unfolds to Quotient.eq, which unfolds to CoPSet.Equiv, which is the cobisimulation relation. The derived theorem of the classical setting is the construction principle of OSetZFA.

Observation 5 (asymmetric embedding proofs). The forward direction of embedPSet_congr (PSet.Equiv → CoPSet.Equiv) is proved by pure bisimulation with no induction; the relation lifting PSet.Equiv through embedPSet is a CoPSet bisimulation. The backward direction (embedPSet_faithful) requires structural induction on PSet's well-foundedness to descend through the coinductive equivalence. Embedding a well-founded structure into a coinductive universe is bisimulation-free; extracting well-founded structure from coinductive equivalence requires well-foundedness as proof resource. The asymmetry reflects the inductive/coinductive divide.

Observation 6 (axiom-free coinductive core). Eight public objects in the cycle are axiom-free: CoPSetFunctor, CoPSet, CoPSet.mk, CoPSet.corec, graphCoalg, graphCoPSet, embedPSet, acc_irrefl. Classical machinery enters only through CoPSet.dest (M-type destructor, via approximation sequences requiring Classical.choice for representative selection) and the bisimulation principles. The coinductive content is constructive; classical machinery is needed only to analyse it, not to construct it.

Observation 7 (definitional dest_corec). PFunctor.M.dest_corec is proved by rfl in mathlib. The computation rule (M.corec f x).dest = ⟨shape, M.corec f ∘ children⟩ is therefore definitional, not requiring simp-based rewriting. Custom coinductive constructions inherit this transparency: graphCoPSet_dest is one-line rfl. This is an engineering benefit of using PFunctor.M as substrate rather than rolling a custom coinductive construction.

Observation 8 (change over simp for notation layers). Definitional equalities crossing multiple notation layers (instance method → typeclass operation → liftOn_2 → underlying function) are not reliably matched by simp only [...]. The kernel-level change tactic, which forces a goal to a specific syntactic form before further tactics apply, succeeds where simp does not. This pattern appeared in mem_mk (Stage 4), graphDecoration_isDecoration (Stage 5), embedOSet_mem (Stage 6), cycleDecoration_eq_quineAtom (Stage 7), and mem_singleton (Stage 8). It is a portable Lean 4 engineering principle for any construction layered through Quotient.lift and typeclass instances.

Observation 9 (Membership typeclass argument order). Lean 4's Membership class has mem : γ → α → Prop with container first, element second. The notation a ∈ b elaborates to inst.mem b a. A custom Membership instance must reverse the natural argument order in the instance body: ⟨fun container element => OSetZFA.Mem element container⟩. Without this reversal, downstream lemmas such as mem_mk become unstatable. Mathlib's ZFSet membership instance follows the same pattern. This is a portable Lean 4 engineering lesson for defining membership on new set-like types.

Observation 10 (parser constraint with doc-comments). Lean 4's set_option attribute cannot appear between a doc-comment and a subsequent declaration. The parser, after closing a doc-comment with -/, expects a declaration keyword (theorem, def, etc.) or an attribute marker, not set_option. Workaround: suppress unused-variable warnings at the binding site (fun _ _ hab => instead of fun a _ hab =>) or place set_option ... in before the doc-comment. Minor but portable lesson for complex declarations.

* * *

Part VII — Position Relative to Neighbouring Formalisations

VII.1 Paulson's Isabelle/HOL work

Paulson (1995–2000s) formalised aspects of non-well-founded set theory in Isabelle/HOL, including hyperset theory and applications to programming language semantics. The approach uses HOL's classical foundations directly, with hypersets constructed as quotients of certain infinite trees. The work predates modern coalgebraic infrastructure in proof assistants.

VR-Sets-ZFA differs in three respects. First, the underlying type theory is dependent (Lean 4) rather than higher-order classical (Isabelle/HOL); CoPSet lives in Type (u+1) parametrised by universe level. Second, the construction substrate is the modern coalgebraic infrastructure (PFunctor.M, the M-type construction of greatest fixpoints) rather than direct tree-based encoding.

Third, VR-Sets-ZFA is positioned within a broader operational-foundations programme (VR Cycle), not as a standalone hyperset theory.

VII.2 Gylterud's HoTT work

Gylterud and collaborators (2018–2025) have formalised non-well-founded sets in homotopy type theory using inductive-recursive constructions and propositional resizing. The HoTT setting allows direct manipulation of trees-with-quotients as h-sets, with AFA derivable from univalence-based reasoning.

VR-Sets-ZFA differs in proof-theoretic strength. HoTT formalisations use univalence and propositional resizing — axioms beyond Lean's standard ceiling. VR-Sets-ZFA operates entirely within [propext, Classical.choice, Quot.sound]; no univalence, no resizing. This makes VR-Sets-ZFA compatible with classical mathlib infrastructure but forfeits the conceptual cleanness of univalent foundations for set identity.

The approaches are complementary. HoTT formalisations capture the categorical content (sets as h-sets with bisimulation as identity) at the cost of additional axiomatic commitment. VR-Sets-ZFA captures the operational content (sets as cobisimulation classes of M-elements) within standard Lean foundations.

VII.3 Classical Aczel

Aczel's original treatment (Non-Well-Founded Sets, 1988) presents AFA as a postulate within set theory, with the universe of sets extended to include non-well-founded objects. Pictures (accessible pointed graphs) decorate this universe via the anti-foundation principle. Bisimulation is derived as the equality relation.

VR-Sets-ZFA inverts the classical presentation: bisimulation is the construction principle, AFA is the derived theorem, the universe is defined through quotient by bisimulation. The operational content is the same; the order of construction is reversed. This inversion is enabled by the coinductive infrastructure: the final coalgebra of the set-shape functor is precisely the universe Aczel postulates, and its universal property is precisely AFA.

VII.4 Position within constructive mathematics

VR-Sets-ZFA is not Bishop-style constructive mathematics. Classical machinery (Classical.choice for representative selection, propext for proof-irrelevance) is used freely within proofs. This matches VR's general two-register apparatus: classical reasoning is permitted in the formal register, with operational content extracted at register boundaries via witnesses.

The operational content is nevertheless real. The eight axiom-free public objects in the cycle — including CoPSet.mk, CoPSet.corec, embedPSet, graphCoalg, graphCoPSet — carry constructive content directly. The classical machinery enters only when analysing structure

through M.dest or bisimulation principles. VR-Sets-ZFA is constructive in content, classical in analytic method.

* * *

References

Aczel, P. (1988). *Non-Well-Founded Sets.* CSLI Publications.

Avigad, J., Carneiro, M. & Hudon, S. (2019). Data types as quotients of polynomial functors. *10th International Conference on Interactive Theorem Proving (ITP 2019)*, Leibniz International Proceedings in Informatics.

Barwise, J. & Moss, L. (1996). *Vicious Circles: On the Mathematics of Non-Wellfounded Phenomena.* CSLI Publications.

Bishop, E. & Bridges, D. (1985). *Constructive Analysis.* Springer.

Gylterud, H. R. (2018). From multisets to sets in homotopy type theory. *Journal of Symbolic Logic*, 83(3), 1132–1146.

Gylterud, H. R., Stenholm, A. & Veltri, N. (2025). Non-wellfounded sets in HoTT/UF. *Logical Methods in Computer Science* (in press).

Paulson, L. C. (1999). Final coalgebras as greatest fixed points in ZF set theory. *Mathematical Structures in Computer Science*, 9(5), 545–567.

Reznik, V. (2026a). VR. A formal system. Zenodo. *DOI:* 10.5281/zenodo.20324391.

Reznik, V. (2026b). VR-Numbers. Zenodo. *DOI:* 10.5281/zenodo.20352239.

Reznik, V. (2026c). VR-Sets. Zenodo. *DOI:* 10.5281/zenodo.20354628.

Reznik, V. (2026d). VR-Forms. Zenodo. *DOI:* 10.5281/zenodo.20355939.

Reznik, V. (2026e). VR-Audit. Zenodo. *DOI:* 10.5281/zenodo.20364111.

Reznik, V. (2026f). VR-Sets-ZFA Lean 4 formalisation, v1.5-vr-sets-zfa. Zenodo. *DOI:* 10.5281/zenodo.20368268.

* * *

Acknowledgements

The work was developed using Claude Opus 4.7 (architectural review) and Claude Sonnet 4.6 (Lean implementation), Variant A (interactive parent-child architecture), consistent with all preceding Lean cycles of the VR Cycle. Version 1.0.1 reframes the exposition to the cycle's no-ontology position: ∅ is taken as a nullary operation and “operational ontology” is renamed the operational universe/foundation; no mathematics is altered. This revision was carried out with Claude Opus 4.8.